VLSI Design Internship — Task 2

Introduction to Verilog HDL and RTL Design of Basic Combinational Circuits

---

Submitted To: Maincrafts Technology

Task: Verilog HDL & RTL Design of Basic Combinational Circuits

Objective

This task documents the design, implementation, and verification of basic combinational logic circuits using Verilog HDL. It covers logic gates (AND, OR, NOT, NAND, NOR, XOR), a Half Adder, and a Full Adder, each described at the Register Transfer Level (RTL) using continuous assignment statements, and verified using self-checking testbenches simulated in Icarus Verilog with waveform analysis performed in GTKWave.

Tools Used

- Icarus Verilog — Verilog compiler and simulator
- GTKWave — waveform viewer for simulation output (dump.vcd)
- EDA Playground — used as an alternate/online simulation environment

1. Logic Gates — Design Logic

Each logic gate is modeled as an independent Verilog module using the **continuous assignment** construct (`assign`), which describes combinational behavior that updates instantaneously whenever an input changes. Each module has two single-bit inputs (A, B — except NOT, which has one input) and a single-bit output Y, connecting inputs to outputs through the corresponding Boolean bitwise operator.

File: logic_gates.v

```
module and_gate (input A, input B, output Y);
assign Y = A & B;
endmodule

module or_gate (input A, input B, output Y);
assign Y = A | B;
endmodule

module not_gate (input A, output Y);
assign Y = ~A;
endmodule

module nand_gate (input A, input B, output Y);
assign Y = ~(A & B);
endmodule

module nor_gate (input A, input B, output Y);
assign Y = ~(A | B);
endmodule

module xor_gate (input A, input B, output Y);
assign Y = A ^ B;
endmodule
```

Expected Truth Table (A, B → Y for each gate)

A	B	AND	OR	NAND	NOR	XOR
0	0	0	0	1	1	0
0	1	0	1	1	0	1
1	0	0	1	1	0	1
1	1	1	1	0	0	0

NOT gate: Y = ~A → A=0 gives Y=1, A=1 gives Y=0.

2. Half Adder — Design Logic

A Half Adder adds two single-bit binary numbers, A and B, producing a Sum and a Carry output. Sum is the XOR of the inputs (true when exactly one input is high), and Carry is the AND of the inputs (true only when both inputs are high — indicating an overflow into the next bit position).

File: half_adder.v

```
module half_adder (
input A,
input B,
output Sum,
output Carry
);
assign Sum = A ^ B;
assign Carry = A & B;
endmodule
```

Truth Table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Testbench: tb_half_adder.v

```
module tb_half_adder;
reg A, B;
wire Sum, Carry;

half_adder uut (.A(A), .B(B), .Sum(Sum), .Carry(Carry));

initial begin
$dumpfile("dump.vcd");
$dumpvars(0, tb_half_adder);

A = 0; B = 0; #10;
A = 0; B = 1; #10;
A = 1; B = 0; #10;
A = 1; B = 1; #10;
$finish;
end
endmodule
```

Simulation Results

The testbench applies all four input combinations of A and B, each held for 10 time units. The waveform in GTKWave confirms Sum and Carry switch exactly at each input transition, matching the truth table above with no unknown (X) or floating (Z) states at any point in the simulation.

- 0,0 → Sum=0, Carry=0
- 0,1 → Sum=1, Carry=0
- 1,0 → Sum=1, Carry=0

- 1,1 → Sum=0, Carry=1
- [Insert GTKWave waveform screenshot here]

3. Full Adder — Design Logic

A Full Adder extends the Half Adder by including a carry-in (Cin), allowing multiple adder stages to be cascaded for multi-bit addition. Sum is the XOR of all three inputs (odd parity), and Cout is high whenever at least two of the three inputs are high, implemented as the OR of the three possible pairwise AND terms.

File: full_adder.v

```
module full_adder (
input A,
input B,
input Cin,
output Sum,
output Cout
);
assign Sum = A ^ B ^ Cin;
assign Cout = (A & B) | (B & Cin) | (A & Cin);
endmodule
```

Truth Table

A	B	Cin	Sum	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Testbench: tb_full_adder.v

```
module tb_full_adder;
reg A, B, Cin;
wire Sum, Cout;

full_adder uut (.A(A), .B(B), .Cin(Cin), .Sum(Sum), .Cout(Cout));

initial begin
$dumpfile("dump.vcd");
$dumpvars(0, tb_full_adder);

A=0; B=0; Cin=0; #10;
A=0; B=0; Cin=1; #10;
A=0; B=1; Cin=0; #10;
A=0; B=1; Cin=1; #10;
```

```
A=1; B=0; Cin=0; #10;
A=1; B=0; Cin=1; #10;
A=1; B=1; Cin=0; #10;
A=1; B=1; Cin=1; #10;
$finish;
end
endmodule
```

Simulation Results

The testbench sweeps through all eight combinations of A, B, and Cin. Sum and Cout track the truth table exactly at every transition, and no unknown or undefined states appear in the waveform output.

- [Insert GTKWave waveform screenshot here]

4. Simulation Flow (Icarus Verilog)

```
# Compile design + testbench
iverilog logic_gates.v half_adder.v tb_half_adder.v

# Run simulation
vvp a.out

# View waveform
gtkwave dump.vcd
```

The same three-step flow (compile → run → view) is repeated for the full adder using its own testbench file.

5. Verification Checklist

- ■ All outputs match their respective truth tables
- ■ No unknown (X) or high-impedance (Z) states observed in any waveform
- ■ Signal transitions occur at the correct simulation time steps
- ■ All modules are written using synthesizable RTL (continuous assignments only)

6. Conclusion

This task provided hands-on experience writing synthesizable Verilog RTL code for basic combinational circuits, developing self-checking testbenches, and verifying functional correctness through simulation and waveform analysis. These skills — module structure, continuous assignment modeling, testbench design, and simulation/debug flow — form the foundation for all subsequent RTL design and verification work in VLSI design.